

Worldline Global – Round 2

SDET / Test Automation Lead

Topic-wise Interview Preparation – Part 2

API Testing • Payment Flows • Reliability • Idempotency • Mocking • Async Processing

How to use this PDF

Read the **layman explanation** first, then the **interview-ready answer**. Focus especially on the highlighted traps and real-world payment scenarios.

Index / Table of Contents

Section	Topic	Page
1	API Failure Handling – HTTP 500	3
2	Retry + Idempotency – Avoiding Duplicate Payments	4
3	Gateway Processed Payment but Response Failed	5
4	API Success but Database Still PENDING	6
5	Synchronous vs Asynchronous Processing	7
6	Senior SDET Thinking – What to Verify	8
7	Last-Minute Revision Sheet	9

Part 2 focus

What happens when a payment system is not perfectly reliable? Think about 500 errors, timeouts, retries, duplicate charges, eventual consistency, database state and reconciliation.

1. API Failure Handling – HTTP 500

Scenario: Your automation expects HTTP 200, but a third-party payment gateway returns HTTP 500.

Layman explanation

HTTP 500 means the server encountered an internal error. As an SDET, checking only the status code is not enough. You need to verify what your application does because of that failure.

Your initial answer

Good starting point

"I will make sure payment not succeeded, proper error handling, idempotency, timeout etc."

Improved interview answer

Interview-ready

"If the third-party payment gateway returns HTTP 500, I would verify that our application handles the failure gracefully. Payment should not be incorrectly marked as successful. I would validate the expected error response, retry behaviour if configured, idempotency, timeout handling, data consistency, and useful logging such as correlation or transaction IDs without exposing sensitive data."

What should you verify?

- **Payment state:** it must not be incorrectly marked SUCCESS.
- **Error contract:** status, error code, message/schema and content type.
- **Retry behaviour:** configured retry count and backoff.
- **Idempotency:** retries must not create duplicate charges.
- **Timeout behaviour:** no indefinite waiting.
- **Data consistency:** payment/order/database state remains valid.
- **Observability:** logs, correlation ID, transaction ID and useful diagnostics.
- **Security:** no sensitive payment data or internal stack traces exposed.

Common trap

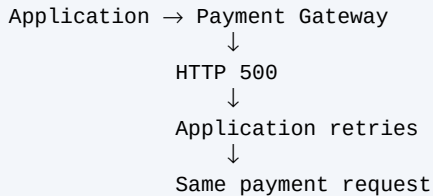
Do not say only: "I will assert `response.statusCode() == 500`." A senior SDET tests the **business impact** of the failure, not just the HTTP number.

2. Retry + Idempotency – Avoiding Duplicate Payments

Interview scenario: The gateway returns HTTP 500 and your application automatically retries the request. How do you verify there is no duplicate payment?

Core idea

A retry is dangerous in payment systems because the first request may have succeeded even when the response was unsuccessful or lost.



What you should verify

- Retry happens only the configured number of times.
- The same **idempotency key / transaction reference** is reused.
- The gateway recognises a duplicate request and does not create another charge.
- Only one successful payment transaction exists.
- The final payment/order status is correct.
- Retry interval/backoff is correct.
- The system eventually stops retrying.

Example

```
Request 1
Idempotency-Key: TX123
↓
Gateway processes ■1000
↓
Response is lost / 500

Retry
Idempotency-Key: TX123
↓
Gateway recognises TX123
↓
No second charge
↓
Original transaction result returned
```

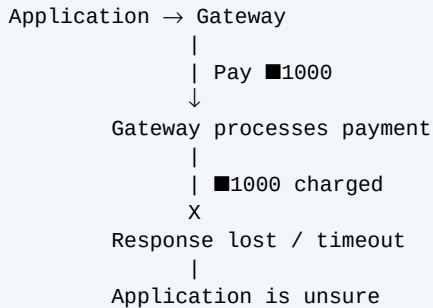
Strong interview phrase

"I would mock the gateway to return 500 initially, trigger the retry mechanism, and verify that the same idempotency key is reused and only one payment transaction is created, even if the first request was actually processed but the response failed."

3. Gateway Process Payment but Response Failed

This is one of the most important payment-domain edge cases.

Scenario



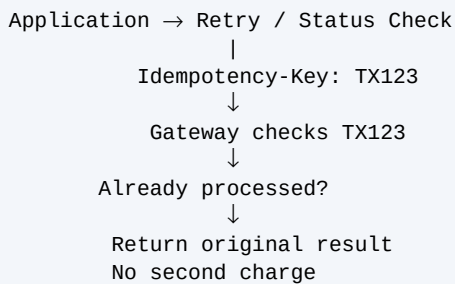
Why is this dangerous?

The application may assume the payment failed and retry. Without idempotency, the customer could be charged twice.

Without idempotency:

1000 charged
1000 charged again
Total = 2000

Correct approach



Interview-ready answer

Senior-level answer

"I would never equate a timeout or 500 response with a definite payment failure. In payment systems, the outcome can be unknown. I would verify idempotency, transaction-status reconciliation, and final consistency between our system and the payment gateway to ensure there is no duplicate charge."

Additional verification

- Use the transaction ID/correlation ID to trace the request.
- Check payment-status API, callback/webhook or reconciliation process.
- Verify exactly one successful gateway transaction.
- Verify our DB/order state eventually reflects the gateway's actual state.
- Verify retries do not create a second transaction.

4. API Success but Database Still PENDING

Scenario:

```
API response:
HTTP 200
{
  "status": "SUCCESS",
  "transactionId": "TX123"
}
```

```
Database:
status = PENDING
```

Do not immediately call this a defect.

First understand whether the payment flow is **synchronous** or **asynchronous**. The system's design determines whether a temporary PENDING state is expected.

Investigation approach

- Understand the expected architecture and consistency model.
- Check whether payment processing is synchronous or asynchronous.
- Check whether DB update happens before or after the API response.
- If asynchronous, verify the state eventually becomes SUCCESS within the defined SLA.
- If synchronous, investigate why the API reported SUCCESS before the DB was updated.
- Compare transaction ID, amount and payment details across API and DB.
- Verify there is no duplicate transaction.
- If DB remains PENDING beyond expected time, capture logs/correlation IDs and fail the test.

Interview-ready answer

Strong answer

"I would first understand whether the payment flow is synchronous or asynchronous. If it is asynchronous, PENDING immediately after the API response may be expected, so I would wait or poll for the final state within the defined SLA and verify that it eventually becomes SUCCESS. If it is synchronous, the mismatch could indicate a data-consistency issue and I would investigate the transaction flow and database update."

5. Synchronous vs Asynchronous Processing

This concept is easy once you connect it to everyday life.

Synchronous = wait for the work to finish

Like ordering a pizza and waiting until the pizza is ready before receiving the final response.

```
Request
↓
Process payment
↓
Update database
↓
Response
```

Asynchronous = start the work and continue

Like placing the pizza order and receiving “Order received” while preparation continues in the background.

```
Request
↓
Start processing
↓
Immediate response
↓
Processing continues
↓
Callback / event
↓
Database updated
```

Comparison

Synchronous	Asynchronous
Caller waits for processing	Caller does not wait for final processing
Response normally follows operation	Response can arrive before completion
DB may already contain final state	DB may temporarily be PENDING
Request → Process → Response	Request → Response → Process

Interview sentence

“I would first understand whether the flow is synchronous or asynchronous. If asynchronous, I would not expect the database to be immediately final; I would validate eventual consistency within the defined SLA.”

6. Senior SDET Thinking – What to Verify

The common pattern behind this section is: **don't stop at the first assertion.**

```
HTTP status
  ↓
Response contract
  ↓
Business state
  ↓
Database state
  ↓
Retries
  ↓
Idempotency
  ↓
Concurrency
  ↓
Observability
  ↓
Final consistency
```

For a payment failure, ask yourself:

- What happened to the money?
- What happened to the transaction?
- What does our application believe happened?
- What does the gateway believe happened?
- Could a retry duplicate the operation?
- Is the database eventually consistent?
- Can we trace the failure using transaction/correlation IDs?

Worldline interview mindset

The selected-candidate feedback emphasised practical problem solving, edge cases, safe concurrency, comprehensive test design, and JUnit/Mockito mocking. Frame answers around **failure modes + business impact + verification**, rather than only syntax or status-code assertions.

What NOT to do

- Do not say every HTTP 500 means payment failed.
- Do not assume API SUCCESS automatically means DB is already SUCCESS.
- Do not say synchronized solves distributed concurrency.
- Do not retry payment blindly without idempotency.
- Do not verify only the HTTP status code.
- Do not hide failures with a broad catch-and-ignore block.

7. Last-Minute Revision Sheet

Memorise these short answers before the interview.

HTTP 500 from gateway

Answer

“Verify correct error handling, payment not marked successful, retry behaviour, idempotency, timeout handling, data consistency and logging.”

Retry causes duplicate payment?

Answer

“Use the same idempotency key/transaction reference and verify only one successful transaction is created.”

Gateway processed payment but response timed out?

Answer

“The outcome is unknown. Do not blindly assume failure. Reconcile the transaction status and use idempotency to prevent duplicate charging.”

API SUCCESS but DB PENDING?

Answer

“First determine whether the flow is synchronous or asynchronous. If asynchronous, validate eventual consistency within the SLA; if synchronous, investigate the inconsistency.”

What is synchronous?

Answer

“The caller waits for the operation to complete before receiving the final response.”

What is asynchronous?

Answer

“The caller can receive a response before processing is complete; the final state may be updated later through an event, callback or background process.”

The golden payment-testing rule

Remember

HTTP response $\neq$ complete business outcome.

Always think about transaction state, retries, idempotency, database consistency and the final outcome.

End of Part 2

Next: asynchronous payment polling, concurrency test automation, advanced mocking, debugging and failure analysis.